

瓦力：服务器对接指南

版本 2025-5-28

1. 概述

本文档供对接瓦力服务器参考，商户可按此文档顺序对接
具体各接口的参数、返回值，可另参考文档《瓦力：服务器接口列表》
前端/App相关问题，可另参考其他文档

瓦力游戏平台，以下简称为“瓦力”
瓦力提供的“API 管理后台”，以下简称为“后台”
商户的玩家，以下称为“用户”

1.1. 对接模式

商户提供的功能

瓦力只提供游戏，以下功能由商户直接向用户提供：

- 用户账号相关功能，如：注册、登录、验证码
- 用户金流相关功能，如：充值、绑卡
- 其他运营功能，如：公告、客服

钱包模式

对接使用“转账钱包模式”，即：

- 用户在商户、瓦力内的余额各自独立。称为“商户钱包”和“游戏钱包”
- 用户在瓦力内，只使用“游戏钱包”，看不到“商户钱包”
- “游戏钱包”初始余额为 0。商户使用瓦力提供的上下分接口在两个钱包间“划拨”余额

瓦力目前仅支持“转账钱包”模式，尚未支持“单一钱包/共享钱包”模式

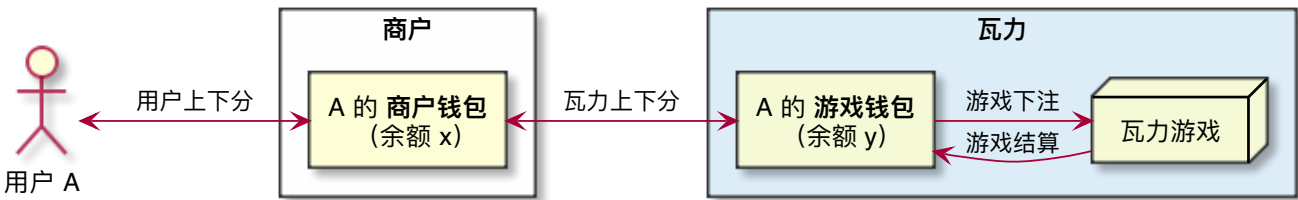


图 1. 转账钱包模式

用户游戏流程

用户玩游戏的完整顺序：

1. 上分：商户调用“上下分接口”，划拨余额到“游戏钱包”
2. 进游戏：商户调用“进游戏接口”，拿到“进游戏链接”。用户使用“进游戏链接”，进入游戏界面
3. 玩游戏：用户在游戏内下注、结算，“游戏钱包”余额随之更新
4. 用户游戏结束，回到商户
5. 下分：商户调用“上下分接口”，取回“游戏钱包”内余额

关于多币种（即将开放）

用户币种默认为 CNY
如商户需要使用多币种，可联系瓦力，将商户配置为“多币种商户”。此时：

- 用户注册、上下分等接口，需指定币种参数
- 一个用户只能使用一个币种，币种无法修改
- 如用户的“商户钱包”支持多币种共存，商户可考虑将每个币种作为单独的用户使用瓦力

1.2. 对接顺序

联调环境

先对接“联调环境”
“联调环境”是包含后台、游戏的完整环境

可按以下流程对接（详见后续章节）：

- [对接：接口加解密](#)
- [对接：上下分流程](#)
- [对接：进游戏流程](#)
- 按需，对接其他接口

正式环境

“联调环境”调试无误后，再开通正式环境账号，对接“正式环境”
“联调环境”与“正式环境”完全隔离，数据不互通
“联调环境”与“正式环境”的“后台地址”、“后台账号”、“接口密钥”均不同

2. 对接：接口加解密

2.1. 对接内容

- 向瓦力提供相关信息，开通“联调环境”账号
拿到“后台地址”、“后台登录账号”，登录后台
- 从后台 [API 与文档 > API 接入信息](#) 页
 - 记录下表信息，供后续调用接口使用
 - （可选）配置 [API-IP白名单](#)
- 按本章后续说明，调用 `ping` 接口
如 `ping` 接口调用成功，可继续 [对接：上下分流程](#)

表 1. 接口调用：需准备信息

信息	含义
API_URL 平台 API 地址	商户使用此地址调用 API 如： https://xxx.com:8080/api
API_ACCOUNT API 账号	仅用于 API 调用。区分大小写 如：3nLfα
AES_KEY 参数加密密钥	业务参数使用 128 位 AES 加密，密钥为 16 字节。使用字符串的 UTF-8 编码作为密钥 如：W@th=w\$BEht3r1aU

信息	含义
SIGN_KEY 请求签名秘钥	纯文本 如：_#tgY\$Q5Xf!74m
AGENT_NAME 商户名	与商户后台登录名一致。用于拼订单号 如：demo

关于 IP 白名单

所有接口都是从 商户 调用 瓦力

如商户未配置 接口IP白名单，则所有 IP 都可以调用接口

如商户配置了 接口IP白名单，则只有白名单内的 IP 才能调用接口，其他 IP 发起的调用会失败

配置 IP 白名单时，不支持通配符，不支持 IP 段

可用半角逗号分隔，配置多个 IP，如 1.2.3.104,1.2.3.105,1.2.3.106

2.2. 请求格式

2.2.1. URL 格式

使用 HTTP GET 发起请求，请求 URL 为：

`${API_URL}/${act}?a=${API_ACCOUNT}&t=${t}&p=${p}&k=${k}`

表 2. URL中变量含义

变量	含义
API_URL	API 地址 如：https://xxx.com:8080/api
act	操作名。区分大小写 如：ping
API_ACCOUNT	API 账号
t	时间戳（Unix时间 / Posix时间，从 1970-1-1 00:00:00 UTC 至现在的秒数） 如：1672531200
p	业务参数，有加密 如：CNyn+Wti8Z73JLZ2R8kNwg==
k	请求签名 如：28659458741870303ab8a3c38700e330

2.2.2. 业务参数 (p)

所有参数拼成一个字符串：

- 拼接方式类似于 url 参数，a=1&b=2&c=3
如（ping 接口）：text=Hello
如（进游戏接口）：uid=123456&ip=1.2.3.4
- 所有接口的参数名、值都不应有特殊符号，拼时无需转义

- 一个参数不应多次出现
- 最终结果中不应有换行符、空格等符号

再将上述字符串进行 AES 加密：

- AES 具体参数为：AES-128、ECB 模式（无 IV）、PKCS#5/PKCS#7 Padding
- 秘钥使用从后台获取的 AES_KEY
- 加密结果使用 Base64 编码



请使用此数据验证

对参数 text=Hello，使用 AES_KEY W@th=w\$BEht3r1aU 加密
结果应为：CNyn+Wti8Z73JLZ2R8kNwg==

参考代码：

```
# 参考代码 (Python)
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad
import base64

AES_KEY = b'W@th=w$BEht3r1aU'
params = 'text=Hello'

cipher = AES.new(AES_KEY, AES.MODE_ECB)
encrypted = cipher.encrypt(pad(params.encode('utf-8'), 16))
p = base64.b64encode(encrypted)
```

PYTHON

```
// 参考代码 (Java)
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.util.Base64;

byte[] AES_KEY = "W@th=w$BEht3r1aU".getBytes(StandardCharsets.UTF_8);
String params = "text=Hello";

Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
cipher.init(Cipher.ENCRYPT_MODE, new SecretKeySpec(AES_KEY, "AES"));
byte[] encrypted = cipher.doFinal(params.getBytes(StandardCharsets.UTF_8));
String p = Base64.getEncoder().encodeToString(encrypted);
```

JAVA

```
// 参考代码 (PHP)
$AES_KEY = 'W@th=w$BEht3r1aU';
$params = 'text=Hello';
$encrypted = openssl_encrypt($params, 'AES-128-ECB', $AES_KEY, OPENSSL_RAW_DATA);
$p = base64_encode($encrypted);
```

PHP

2.2.3. 请求签名 (k)

签名计算方式为：

- 将 p、t、SIGN_KEY 拼接为一个字符串
- 对此字符串计算 MD5
- MD5 结果输出为 16 进制（即，Hex 格式）



请使用此数据验证

对 `p="CNyn+Wti8Z73JLZ2R8kNwg=="`、`t=1672531200`、`SIGN_KEY="_#tgY$Q5Xf!74m"`
结果应为：`28659458741870303ab8a3c38700e330`

参考代码：

```
# 参考代码 (Python)
import hashlib

src = f'{p}{t}{SIGN_KEY}'
k = hashlib.md5(src.encode('utf-8')).hexdigest()
```

PYTHON

```
// 参考代码 (Java)
import org.apache.commons.codec.binary.Hex;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;

String src = p + t + SIGN_KEY;
MessageDigest digest = MessageDigest.getInstance("MD5");
byte[] md5 = digest.digest(src.getBytes(StandardCharsets.UTF_8));
String k = Hex.encodeHexString(md5);
```

JAVA

2.2.4. 拼装 URL

按此方式拼装：`${API_URL}/${act}?a=${API_ACCOUNT}&t=${t}&p=${p_encoded}&k=${k}`

需注意，由于 `p` 中可能有特殊符号（`/ + =`），在拼装 URL 时应转义处理

如后续示例中，`+` 在 URL 中已转为 `%2B`

如使用工具库拼 URL，工具库可能已自带此处理，不应重复转义

请使用此数据验证

对 `API_URL="https://xxx.com:8080/api"`、`act="ping"`、
`API_ACCOUNT="3nLfa"`、`t=1672531200`、
`p="CNyn+Wti8Z73JLZ2R8kNwg=="`、`k="28659458741870303ab8a3c38700e330"`
最终 URL 应为：`https://xxx.com:8080/api/ping?a=3nLfa&t=1672531200&p=CNyn%2BWti8Z73JLZ2R8kNwg%3D%3D&k=28659458741870303ab8a3c38700e330`

2.3. 返回格式

返回为 JSON，明文无加密

成功返回如：`{"code":0,"data":{"text":"Hello"}}`

异常返回如：`{"code":601,"msg":"illegal_act"}`

表 3. 返回 JSON 字段含义

字段	含义
code	状态码。一定有此字段，否则理解为返回异常 正常情况下，code 应为 0，通过 data 返回业务处理结果 如 code 为其他值，表示处理时遇到问题

字段	含义
msg	供调试的详细信息。仅失败时有此字段 供诊断异常时人工查阅，可能会调整，不应直接展示给用户，不应用于程序判断
data	返回的业务数据（类型为 JSON object） 如： <code>{"text":"Hello"} , {"gameUrl":"https://xxxxxx"}</code> 《瓦力：服务器接口列表》中的“返回字段”指本字段内的字段

表 4. 返回字段：常见 code

code	含义
0	正常处理
600	业务参数有误 如：缺少业务参数、业务参数格式有误
601	请求鉴权未通过 如：加解密、签名、IP 白名单 相关问题
629	请求过于频繁而被拦截
700	瓦力内部处理时遇到问题

表 5. 返回字段：常见 msg

返回	含义
<code>{"code":601,"msg":"illegal_a:..."} </code>	<code>a</code> 参数异常。请检查“API 账号”是否正确 “API 账号”不是后台登录账号，仅用于 API 调用 联调环境与正式环境的账号不同
<code>{"code":601,"msg":"illegal_p:base64:..."} </code>	<code>p</code> 参数异常：不是有效的 Base64 建议检查：拼装 URL 时， <code>p</code> 参数是否有正确的 URL 转义 (Base64 结果可能有 <code>/ + =</code> 符号)
<code>{"code":601,"msg":"illegal_p:aes"} </code>	<code>p</code> 参数异常：AES 解密失败 建议检查相关代码，并使用 业务参数 (p) 小节示例数据验证 建议检查：使用的 AES_KEY 是否正确（联调环境与正式环境不同）
<code>{"code":601,"msg":"illegal_t"} </code>	<code>t</code> 参数异常 正常情况下， <code>t</code> 参数与当前时间，偏差不应超过 1 分钟 建议检查：时间格式是否正确，时间单位为 <code>秒</code> ，非毫秒 建议检查：服务器时间。Linux 系统可使用 <code>ntpdate</code> 修正系统时间

返回	含义
<code>{"code":601,"msg":"illegal_ip:..."} </code>	IP 拦截 建议检查：后台配置的 IP 白名单 是否符合预期 如刚调整过配置，需约 2 分钟 生效
<code>{"code":601,"msg":"illegal_k"} </code>	k 参数异常。签名不对 建议检查相关代码，并使用 请求签名 (k) 小节示例数据验证 建议检查：使用的 SIGN_KEY 是否正确（联调环境与正式环境不同）
<code>{"code":600,"msg":"illegal_act:..."} </code>	接口名无效
<code>{"code":600,"msg":"xxx_required"} </code>	缺少业务参数 xxx
<code>{"code":600,"msg":"xxx_illegal"} </code>	业务参数 xxx 格式不对
<code>{"code":629,"msg":"too_many_requests"} </code>	接口调用过于频繁，应降低请求频率 各接口限制不同，联调环境与正式环境限制不同
<code>{"code":700,"msg":"internal_error"} </code>	瓦力内部错误，请联系瓦力

2.4. 接口文档

各接口的参数、返回字段，可参考文档《瓦力：服务器接口列表》

《瓦力：服务器接口列表》内，为保持简明清晰，每个接口：

- 标题中的英文即为操作名 `act`，如上下分的操作名为 `transferV3`
- [请求参数](#) 一节，为所有的业务参数（通过 URL `p` 参数传递）
- [返回字段](#) 一节，对应返回 JSON 中的 `data` 内部字段

必须接入的接口有：

- 上下分相关：`transferV3` 上下分、`queryOrderV3` 查单、`getBalance` 查用户是否可下分
- 进游戏相关：`enterGame` 进游戏
- 其他接口商户可根据需求选择是否对接

3. 对接：上下分流程

3.1. 对接内容

使用户能将余额在“商户钱包”与“瓦力钱包”间划拨

相关接口主要是：`transferV3` 上下分、`queryOrderV3` 查单，详细参数见《瓦力：服务器接口列表》

支持余额划拨后，可继续 [对接：进游戏流程](#)

3.2. 处理细节

对上下分，有以下细节需注意：

商户的操作顺序

- 上分操作
为避免“商户钱包”不够扣，商户应先扣“商户钱包”，扣款成功后再调用瓦力接口上分
如商户钱包扣款失败，则取消上分
如商户钱包扣款成功，继续调用瓦力接口失败，则归还商户钱包扣款
- 下分操作
为避免“没下成功的分被花出去”，商户应先调用瓦力接口，成功下分后再更新商户钱包余额

调用上下分接口，结果的处理

- 如由于网络超时，未拿到返回结果
商户可重发，也可间隔一段时间调用查单接口（建议至少间隔 2 分钟）
如商户重发，对同一单号，瓦力不会重复处理，会返回一致结果（即，幂等）
如商户重发，建议最多重发一次。如经常超时，应检查网络环境、调用配置等
除超时外，其他情况不应重发。如：接口鉴权问题、请求参数问题、用户余额问题等，重发没用
- 如返回上下分“成功”或“失败”（status: 1,2）
此状态即为最终结果，不会再更新
- 如返回上下分“处理中”（status: 0）
说明瓦力尚未处理完。商户需在一段时间后，调用查单接口确认状态
此状态正常情况下极少出现。可能的原因是：商户并发请求、瓦力内部系统更新

调用查单接口，结果的处理

- 如返回“未找到”（status: -1）
瓦力收到上下分操作后，如参数有效，会先入库再处理，入库后即可查到
如上下分调用后立即查询，可能会由于此单“正在入库”而查不到，过一会儿再查就能查到了
如上下分最后一次调用已超过 5 分钟，查询仍返回“未找到”，可理解为瓦力确实没有此单
- 如返回上下分“处理中”（status: 0）
建议商户持续以较长间隔（20分钟以上）查询，直到最终状态变为“成功”或“失败”

瓦力内部，上下分的处理流程如图，供参考：

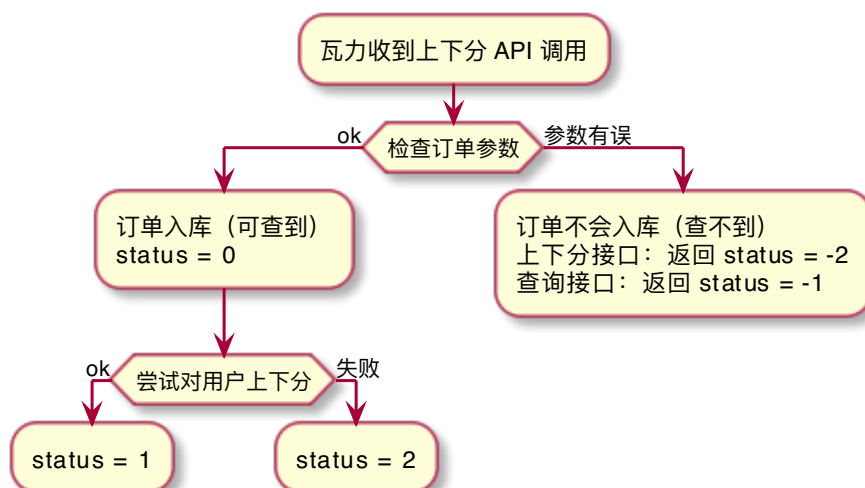


图 2. 上下分：瓦力内部处理流程

游戏对上下分的影响

- 大部分游戏，用户玩游戏不影响上下分。随上下分操作，用户“瓦力钱包”的更新会即时更新到用户游戏界面

- 对少量棋牌对战类游戏，由于局后结算，或有相关“防以小博大”规则，用户在这些游戏内时，限制上下分
此时发起的上下分会立即失败，返回 `status: 2, reason: "game_limit"`
具体游戏如：斗地主、抢庄牛牛等
- 商户受影响较大的是用户退出后的下分流程
需注意，即使用户关闭游戏界面，仍有可能由于本局游戏未结束而无法下分
此时商户可通过接口 `getBalance` 查询用户当前是否可以下分

3.3. 参考流程

综上，对上下分操作，商户可参考以下流程，结合自己需求调整、优化

```
# 伪码

def 上分流程:
    创建订单记录 # 订单记录状态为 "新创建"
    用户商户钱包 扣除余额 # 如扣除失败，停止划拨

    调用瓦力接口 上下分
    if 接口返回状态为 "成功":
        更新订单记录状态为 "瓦力划拨成功"
        return 划拨成功
    else if 接口返回状态为 "失败":
        更新订单记录状态为 "瓦力划拨失败"
        用户商户钱包 退回余额
        更新订单记录状态为 "已退回用户余额"
        return 划拨失败
    else:
        订单加入异常订单队列 # 后续定时重试
        return 划拨处理中

def 下分流程:
    创建订单记录 # 订单记录状态为 "新创建"

    调用瓦力接口 上下分
    if 接口返回状态为 "成功":
        更新订单记录状态为 "瓦力划拨成功"
        用户商户钱包 增加余额
        更新订单记录状态为 "已增加用户余额"
        return 划拨成功
    else if 接口返回状态为 "失败":
        更新订单记录状态为 "瓦力划拨失败"
        return 划拨失败
    else:
        订单加入异常订单队列 # 后续定时重试
        return 划拨处理中
```

PYTHON

4. 对接：进入游戏流程

4.1. 对接内容

使用户能进瓦力游戏
相关接口：enterGame

4.2. 处理细节

4.2.1 用户监控亡卡

4.2.1. 用户进入游戏

调用 进入游戏接口，返回的 URL 中有用户的“瓦力 token”
“瓦力 token”用于瓦力游戏识别用户身份，有效期 10 天
如用户在游戏内，还会持续延长有效期

“瓦力 token”在每次调用 进入游戏接口 时重置
如多次调用 进入游戏接口，只有最后一次返回的 URL 可用，之前的 URL 会因 token 被重置而无法使用

4.2.2. 用户退出方式

如商户是 App，通过 WebView 打开瓦力游戏
建议在 WebView 上层加一个“悬浮按钮”
点击时，关闭 WebView，返回商户界面
(悬浮按钮也可考虑提供更多功能，如充值)

如商户是 Web，瓦力支持使用 iframe 嵌入

4.2.3. 关于用户游戏状态

用户打开某款游戏的界面，加载完成，显示游戏画面后，认为用户“在这个游戏中”
用户退出游戏界面，且游戏内已全部结算完毕，认为用户“已离开此游戏”
如游戏有选房间页面，选房间页面不算游戏内，进入某一房间画面后，认为用户“在这个游戏中”

如用户在 A 游戏中，用户进 B 游戏时，提示：
您当前还在其他游戏中，是否立刻回到该游戏

4.2.4. 常见用户提示

表 6. 用户UI常见提示

提示	含义
账号认证已失效，请关闭游戏后重新登录	“瓦力 token” 已失效 需重新调用进入游戏接口，使用最新的 URL
您当前还在其他游戏中，是否立刻回到该游戏	多款游戏不能同时使用 用户 A 游戏一局尚未结束，此时进入 B 游戏，会有此提示
您的账号已在其他地方登录，请重新登录	游戏界面不能多开 如新开游戏界面，原有界面被顶掉，会有此提示